

IA + Verificação Formal para Auditoria e Correção Automática

Gustavo Ferreira Leite de Barros — Centro de Informática, UFPE

Pipeline: Slither → LLM → Certora Prover → diagnóstico → correção → revalidação

Contratos que movem dinheiro, e podem falhar

- Contratos inteligentes automatizam transferências financeiras sem intermediário — uma falha no código pode ser explorada e causar prejuízo financeiro real.
- Desenvolvedores também otimizam contratos para reduzir o custo de gás — mas alterar o código pode mudar seu comportamento original ou introduzir novas vulnerabilidades.
- Testes tradicionais só cobrem alguns exemplos: não provam que o contrato está correto para todos os casos possíveis.

A SOLUÇÃO

Uma pipeline automática de análise e correção



A IA não corrige "no chute": ela opera dentro de um pipeline com análise estática, verificação formal e um ciclo de revalidação.

Slither + LLM

- O **Slither** examina o código estaticamente e aponta possíveis vulnerabilidades (controle de acesso, endereços não validados, chamadas externas inseguras, entre outras).
- Um modelo de IA (**Llama 3.3 70B**) interpreta esses achados, identifica quais são relevantes para o contrato e ajuda a criar propriedades de segurança formalizáveis.

Certora Prover: prova, não amostra

- As propriedades viram uma **especificação CVL** e são analisadas pelo Certora Prover.
- Diferente de testes tradicionais, que avaliam alguns exemplos, a verificação formal analisa **matematicamente** todos os comportamentos possíveis do contrato.
- Quando uma propriedade é violada, a ferramenta devolve um **contraexemplo** concreto — a entrada exata que quebra a garantia.

Diagnóstico → correção → revalidação

- Com o contraexemplo em mãos, a IA diagnostica a causa raiz da falha e produz uma versão corrigida do contrato.
- A correção **não é aceita de imediato**: o contrato corrigido passa de novo pelo Slither e pelo Certora.
- Se a violação persistir, o sistema tenta de novo (até 3 vezes) — um ciclo automático e verificável, não um patch "no chute".

ESTUDO DE CASO

DeFiVault — cofre de finanças descentralizadas

Contrato pequeno (187 linhas) com controle de acesso, endereços não validados, transferência insegura e risco de reentrância.

ANTES — tx.origin como autorização

```
modifier onlyOwner() {  
    require(tx.origin == owner,  
        "not owner");  
    _;  
}
```

ANTES — endereço zero aceito

```
function transferOwnership(  
    address newOwner  
) external onlyOwner {  
    pendingOwner = newOwner;  
}
```

DEPOIS — msg.sender (correto)

```
modifier onlyOwner() {  
    require(msg.sender == owner,  
        "not owner");  
    _;  
}
```

DEPOIS — validação adicionada

```
function transferOwnership(  
    address newOwner  
) external onlyOwner {  
    require(newOwner != address(0));  
    pendingOwner = newOwner;  
}
```

RESULTADO

5 violações confirmadas → 5 corrigidas, em 1 iteração

ID	Tipo	Função	Status
VULN_003	missing-zero-check	transferOwnership	Resolvido
VULN_004	missing-zero-check	emergencyWithdraw	Resolvido
VULN_011	tx-origin	onlyOwner	Resolvido
VULN_012	suicidal	destroy	Resolvido
VULN_013	arbitrary-send-eth	emergencyWithdraw	Resolvido

Correção aplicada em **withdraw** e **claimReward**: padrão checks-effects-interactions (estado atualizado antes da chamada externa), reduzindo a superfície de reentrância — hardening adicional além das 5 regras verificadas no Certora.

Geração + prova, não geração sozinha

- O diferencial é combinar a **capacidade de geração** da IA com as **evidências da verificação formal**.
- Objetivo: tornar a auditoria de contratos inteligentes mais automática, segura e confiável — não substituir a auditoria humana, mas reduzir o trabalho repetitivo.
- A correção só é aceita depois de confirmada matematicamente, e ainda passa por um guardrail de minimalidade antes da revalidação.

Obrigado!